

Financial Data Analysis Using SQL

Project Report

A SQL-driven analysis of income, expenses, savings, budgets,
and account activity across a transactional financial dataset

Database	: finance_db (MySQL)
Tables Used	: accounts, transactions, categories
Total Records Analyzed	: 2,000
Business Questions Solved	: 20

Contents

1	Introduction	2
2	Database Schema	2
3	Tools & Techniques Used	2
4	Business Problems & SQL Solutions	3
5	Key Results	9
6	Key Insights	10
7	Conclusion	10

1 Introduction

Personal and business finance generates large volumes of transactional data every month, but raw transaction logs on their own rarely answer the questions that matter: How much am I earning? How much am I spending? Am I saving enough? Where exactly is the money going?

This project uses **SQL (MySQL)** to convert a raw financial transactions dataset into clear, decision-ready answers. Rather than relying on a BI tool or dashboard software, all analysis — aggregation, trend calculation, budget comparison, and running balances — is performed directly through structured SQL queries.

Objective

To analyze a financial database and answer 20 real-world business questions covering income, expenses, savings, budgeting, account balances, spending trends, and overall financial health.

2 Database Schema

The analysis is built on three interlinked tables inside the `finance_db` database.

Table	Description
<code>accounts</code>	Stores account details — account ID, name, type, and opening balance
<code>transactions</code>	Stores individual transactions — amount, date, account ID, category ID
<code>categories</code>	Stores category names and transaction type (Income / Expense)

Dataset Size

- Total Transaction Records: **2,000**
- Active Accounts: **1,254**

3 Tools & Techniques Used

- **Database:** MySQL
- **Joins:** INNER JOIN across transactions, categories, and accounts
- **Aggregate Functions:** SUM, COUNT, AVG, MAX
- **CASE Statements:** Conditional aggregation for Income vs. Expense splits
- **CTEs (Common Table Expressions):** Multi-step logic for budget variance and recurring expense detection
- **Window Functions:** LAG () for growth rate, SUM () OVER () for running balances
- **Date Functions:** DATE_FORMAT () for monthly grouping, DAYOFWEEK () for weekday/weekend analysis
- **Subqueries:** Used in budget comparison and category contribution calculations

4 Business Problems & SQL Solutions

This section presents each business question along with the SQL query used to answer it.

Q1. What is the total income across all accounts?

```
SELECT SUM(ABS(transactions.amount)) AS Total_Income
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = "Income";
```

Q2. What is the total amount of expenses across all accounts?

```
SELECT SUM(ABS(transactions.amount)) AS Total_Amount
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = "Expense";
```

Q3. What is the net savings (total income minus total expenses)?

```
SELECT
  ROUND(SUM(CASE WHEN categories.transaction_type = "Income"
    THEN ABS(transactions.amount) ELSE 0 END), 2) AS Total_Income,
  ROUND(SUM(CASE WHEN categories.transaction_type = "Expense"
    THEN ABS(transactions.amount) ELSE 0 END), 2) AS Total_Expenses,
  (SUM(CASE WHEN categories.transaction_type = "Income"
    THEN ABS(transactions.amount) ELSE 0 END) -
    SUM(CASE WHEN categories.transaction_type = "Expense"
    THEN ABS(transactions.amount) ELSE 0 END)) AS Net_Saving
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id;
```

Q4. What percentage of income is being saved (savings rate)?

```
SELECT
  ROUND(SUM(CASE WHEN categories.transaction_type = "Income"
    THEN ABS(transactions.amount) ELSE 0 END), 2) AS Total_Income,
  ROUND(SUM(CASE WHEN transaction_type = "Expense"
    THEN ABS(transactions.amount) ELSE 0 END), 2) AS Total_Expenses,
  ROUND((SUM(CASE WHEN categories.transaction_type = "Income"
    THEN ABS(transactions.amount) ELSE 0 END) -
    SUM(CASE WHEN categories.transaction_type = "Expense"
    THEN ABS(transactions.amount) ELSE 0 END)) * 100.0 /
    NULLIF(SUM(CASE WHEN categories.transaction_type = "Income"
    THEN ABS(transactions.amount) ELSE 0 END), 0), 2) AS Saving_Rate_Percent
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id;
```

Q5. What is the monthly cash flow (income vs expenses per month)?

```
SELECT
  DATE_FORMAT(transactions.transaction_date, '%Y-%m') AS Month_Wise,
  SUM(CASE WHEN categories.transaction_type = "Income"
    THEN ABS(transactions.amount) ELSE 0 END) AS Total_Income,
  SUM(CASE WHEN categories.transaction_type = "Expense"
```

```

        THEN ABS(transactions.amount) ELSE 0 END) AS Total_Expenses,
    ROUND(SUM(CASE WHEN categories.transaction_type = "Income"
        THEN ABS(transactions.amount) ELSE -ABS(transactions.amount) END), 2) AS
        Monthly_Cash_Flow
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
GROUP BY Month_Wise
ORDER BY Month_Wise
LIMIT 12;

```

Q6. How much income is coming from each income source/category?

```

SELECT categories.category_name AS Income_Source,
    SUM(ABS(transactions.amount)) AS Total_Amount
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = 'Income'
GROUP BY Income_Source
ORDER BY Total_Amount DESC
LIMIT 10;

```

Q7. How much is being spent in each expense category?

```

SELECT categories.category_name AS Category_Name,
    SUM(ABS(transactions.amount)) AS Total_Expense
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = 'Expense'
GROUP BY Category_Name
ORDER BY Total_Expense DESC
LIMIT 10;

```

Q8. Which expense category has the highest total spending?

```

SELECT categories.category_name AS Category_Name,
    SUM(ABS(transactions.amount)) AS Total_Amount
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = "Expense"
GROUP BY Category_Name
ORDER BY Total_Amount DESC
LIMIT 1;

```

Q9. How does actual spending compare to the budgeted amount for each category?

```

WITH Monthly AS (
    SELECT
        DATE_FORMAT(transactions.transaction_date, '%Y-%m') AS Monthly_Wise,
        categories.category_name AS Category_Name,
        SUM(ABS(transactions.amount)) AS Spend_Amount
    FROM transactions
    JOIN categories ON categories.category_id = transactions.category_id
    WHERE categories.transaction_type = "Expense"
    GROUP BY Category_Name, Monthly_Wise
),
budget AS (
    SELECT Category_Name, ROUND(AVG(Spend_Amount), 2) AS Budget_Amount
    FROM Monthly

```

```

    GROUP BY Category_Name
),
latest_Month AS (
    SELECT MAX(Monthly_Wise) AS m FROM Monthly
),
Actual AS (
    SELECT Category_Name, ROUND(Spend_Amount, 2) AS Actual_Amount
    FROM Monthly, latest_Month
    WHERE Monthly.Monthly_Wise = latest_Month.m
)
SELECT
    budget.Category_Name,
    budget.Budget_Amount,
    COALESCE(Actual.Actual_Amount, 0) AS Actual_Amount,
    COALESCE(Actual.Actual_Amount, 0) - budget.Budget_Amount AS Variance
FROM budget
LEFT JOIN Actual ON Actual.Category_Name = budget.Category_Name
ORDER BY Variance DESC
LIMIT 10;

```

Q10. What is the average amount spent per day?

```

SELECT ROUND(SUM(ABS(amount)) * 1.0 / COUNT(DISTINCT transaction_date), 2)
    AS Avg_Daily_Spending
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = 'Expense';

```

Q11. What is the trend of total spending month by month?

```

SELECT
    DATE_FORMAT(transactions.transaction_date, '%Y-%m') AS Month_Wise,
    SUM(ABS(transactions.amount)) AS Spending_Amount
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = 'Expense'
GROUP BY Month_Wise
ORDER BY Month_Wise
LIMIT 12;

```

Q12. What is the running account balance after each transaction?

```

WITH top_account AS (
    SELECT account_id FROM transactions
    GROUP BY account_id
    ORDER BY COUNT(*) DESC
    LIMIT 1
)
SELECT
    transactions.transaction_id,
    transactions.transaction_date,
    transactions.amount AS Amount,
    accounts.opening_balance + SUM(transactions.amount)
        OVER (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS Running_Balance
FROM transactions
JOIN accounts ON accounts.account_id = transactions.account_id
WHERE transactions.account_id = (SELECT account_id FROM top_account)
ORDER BY transactions.account_id, transactions.transaction_date
LIMIT 10;

```

Q13. Which expenses are recurring every month (subscriptions, bills, etc.)?

```

SELECT
    categories.category_name AS Category_Name,
    COUNT(DISTINCT (DATE_FORMAT(transactions.transaction_date, '%Y-%m'))) AS
        Months_Active,
    SUM(ABS(transactions.amount)) AS Total_Spent,
    ROUND(AVG(ABS(transactions.amount)), 2) AS Avg_Spent
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = 'Expense'
GROUP BY Category_Name
HAVING Months_Active >= 6
ORDER BY Months_Active DESC, Total_Spent DESC
LIMIT 10;

```

Q14. Is spending higher on weekends or on weekdays?

```

SELECT
    CASE WHEN DAYOFWEEK(transactions.transaction_date) IN (1, 7)
    THEN 'Weekend' ELSE 'Weekday' END AS Day_Type,
    COUNT(*) AS Num_Transactions,
    SUM(ABS(transactions.amount)) AS Total_Spent,
    ROUND(AVG(ABS(transactions.amount)), 2) AS Avg_Spent
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = 'Expense'
GROUP BY Day_Type
ORDER BY Num_Transactions DESC;

```

Q15. Which are the top 10 largest transactions overall?

```

SELECT
    transactions.transaction_id,
    transactions.transaction_date,
    accounts.account_name,
    categories.category_name,
    ABS(transactions.amount) AS Total_Amount
FROM transactions
JOIN accounts ON accounts.account_id = transactions.account_id
JOIN categories ON categories.category_id = transactions.category_id
ORDER BY Total_Amount DESC
LIMIT 10;

```

Q16. What is the current balance of each account?

```

SELECT
    accounts.account_id,
    accounts.account_name,
    accounts.account_type,
    accounts.opening_balance,
    (accounts.opening_balance + COALESCE(SUM(transactions.amount), 0)) AS
        Current_Balance
FROM transactions
JOIN accounts ON accounts.account_id = transactions.account_id
GROUP BY accounts.account_id, accounts.account_name, accounts.account_type, accounts.
    opening_balance
ORDER BY Current_Balance DESC
LIMIT 10;

```

Q17. How much is income growing or shrinking month over month?

```

WITH monthly AS (
    SELECT
        DATE_FORMAT(transactions.transaction_date, '%Y-%m') AS Month_Wise,
        SUM(ABS(transactions.amount)) AS Income
    FROM transactions
    JOIN categories ON categories.category_id = transactions.category_id
    WHERE categories.transaction_type = 'Income'
    GROUP BY Month_Wise
)
SELECT
    Month_Wise,
    Income,
    ROUND((Income - LAG(Income) OVER (ORDER BY Month_Wise)) * 100 /
        NULLIF(LAG(Income) OVER (ORDER BY Month_Wise), 0), 2) AS Growth_Rate
FROM monthly
ORDER BY Month_Wise;

```

Q18. How much are expenses growing or shrinking month over month?

```

WITH monthly AS (
    SELECT
        DATE_FORMAT(transactions.transaction_date, '%Y-%m') AS Month_Wise,
        SUM(transactions.amount) AS Expense
    FROM transactions
    JOIN categories ON categories.category_id = transactions.category_id
    WHERE categories.transaction_type = 'Expense'
    GROUP BY Month_Wise
)
SELECT
    Month_Wise,
    Expense,
    (Expense - LAG(Expense) OVER (ORDER BY Month_Wise)) * 100 /
    NULLIF(LAG(Expense) OVER (ORDER BY Month_Wise), 0) AS Growth_Rate
FROM monthly
ORDER BY Month_Wise;

```

Q19. What percentage does each category contribute to total spending?

```

WITH monthly AS (
    SELECT SUM(ABS(transactions.amount)) AS Total_Expense
    FROM transactions
    JOIN categories ON categories.category_id = transactions.category_id
    WHERE categories.transaction_type = 'Expense'
)
SELECT
    categories.category_name AS Category_Name,
    SUM(ABS(transactions.amount)) AS Category_Expense,
    ROUND((SUM(ABS(transactions.amount)) * 100 / (SELECT Total_Expense FROM monthly)),
        2)
    AS Percentage_Contribution
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id
WHERE categories.transaction_type = 'Expense'
GROUP BY Category_Name
ORDER BY Category_Expense DESC
LIMIT 10;

```

Q20. What does the overall financial health dashboard look like? (Summary of key numbers)

```
SELECT
  SUM(CASE WHEN categories.transaction_type = 'Income'
    THEN ABS(transactions.amount) ELSE 0 END) AS Total_Income,
  SUM(CASE WHEN categories.transaction_type = 'Expense'
    THEN ABS(transactions.amount) ELSE 0 END) AS Total_Expense,
  SUM(CASE WHEN categories.transaction_type = 'Income'
    THEN ABS(transactions.amount) ELSE 0 END) -
  SUM(CASE WHEN categories.transaction_type = 'Expense'
    THEN ABS(transactions.amount) ELSE 0 END) AS Net_Savings,
  (SUM(CASE WHEN categories.transaction_type = 'Income'
    THEN ABS(transactions.amount) ELSE 0 END) -
  SUM(CASE WHEN categories.transaction_type = 'Expense'
    THEN ABS(transactions.amount) ELSE 0 END)) * 100 /
  NULLIF(SUM(CASE WHEN categories.transaction_type = 'Income'
    THEN ABS(transactions.amount) ELSE 0 END), 0) AS Savings_Rate_Percentage,
  COUNT(DISTINCT(transactions.transaction_id)) AS Total_Transactions,
  COUNT(DISTINCT(transactions.account_id)) AS Active_Accounts
FROM transactions
JOIN categories ON categories.category_id = transactions.category_id;
```

5 Key Results

The final dashboard query (Q20) was executed against the full dataset. The output is summarized below.

Metric	Value
Total Records in Dataset	2,000
Total Income	Rs. 14,81,602.09
Total Expenses	Rs. 15,08,106.99
Net Savings	–Rs. 26,504.90
Savings Rate	–1.79%
Total Transactions	2,000
Active Accounts	1,254

6 Key Insights

- **Negative Savings Rate:** Total expenses (Rs. 15.08L) marginally exceeded total income (Rs. 14.82L) across the analyzed period, producing a net savings of –Rs. 26,504.90 and a savings rate of –1.79%. This indicates that, in aggregate, outflows are slightly outpacing inflows — a warning sign that would typically prompt a closer review of discretionary spending categories.
- **Monthly Cash Flow Visibility:** Grouping income and expenses by month (Q5) makes it possible to pinpoint exactly which months tipped into deficit, rather than relying on a single aggregate figure.
- **Category Concentration:** A small number of expense categories account for a disproportionate share of total spending (Q7, Q8, Q19), making them the most impactful targets for cost control.
- **Budget Variance:** Comparing the latest month's actual spend against each category's historical average (Q9) highlights categories that are currently overshooting their typical budget.
- **Recurring Expenses:** Categories active in 6 or more months (Q13) behave like fixed recurring costs (subscriptions, bills, rent) and represent the most predictable — and most negotiable — portion of monthly outflow.
- **Spending Behavior:** Comparing weekday vs. weekend transactions (Q14) reveals whether discretionary spending clusters around leisure time, which is useful for habit-based budgeting advice.
- **Account-Level View:** Running balances (Q12) and current balances by account (Q16) give a transaction-by-transaction and account-by-account picture, not just a single organization-wide number.
- **Growth Trends:** Month-over-month growth rates for income and expenses (Q17, Q18) show whether the gap between earning and spending is widening or narrowing over time.

7 Conclusion

This project demonstrates that a complete personal/business finance analytics workflow — covering income and expense tracking, savings rate, monthly trends, budget variance, recurring cost detection, and account balances — can be built entirely with structured SQL queries, without any dedicated BI tool.

The analysis surfaced an important finding: expenses are currently exceeding income by Rs. 26,504.90, resulting in a negative savings rate. This is the kind of insight a systematic SQL-based review is designed to catch early, before a small monthly shortfall compounds into a larger financial problem.

Future Scope:

- Visualize the same queries in Power BI / Excel for a stakeholder-facing dashboard.
- Extend the budget variance check to run automatically for every month rather than just the latest one.
- Add anomaly detection to flag unusually large or unusual transactions in real time.